

Short recap day #2

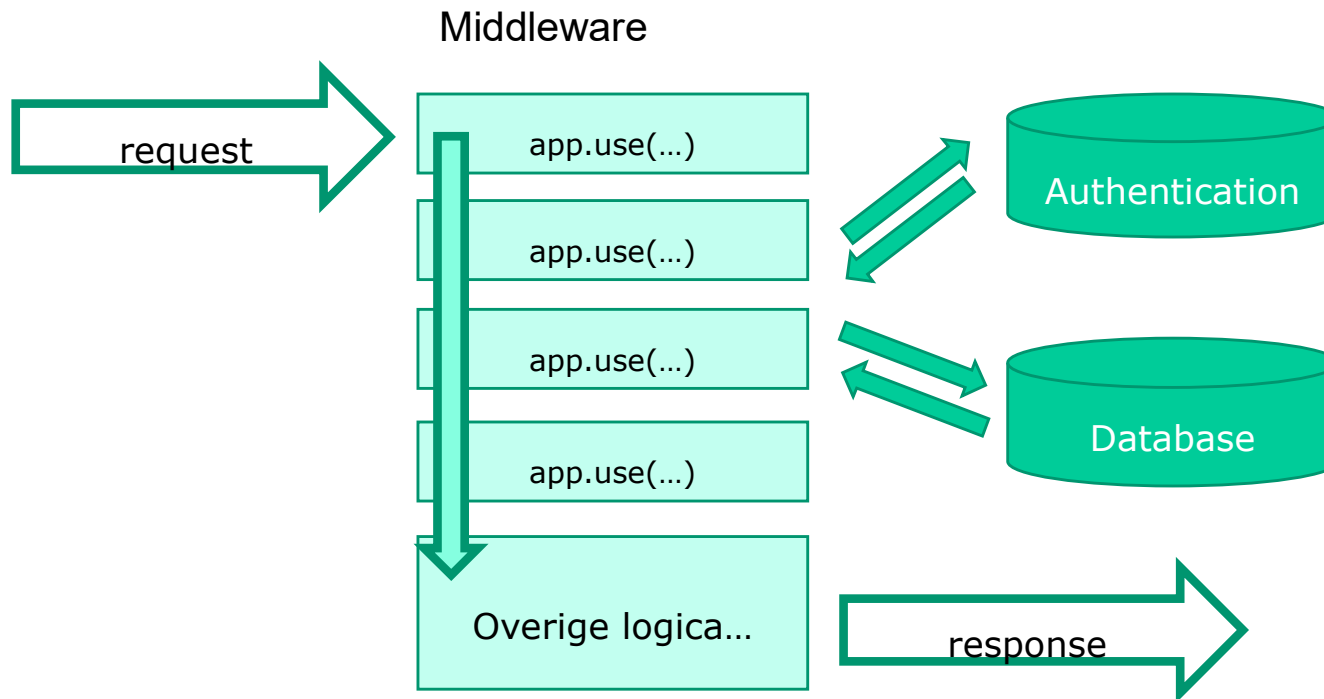


Peter Kassenaar – info@kassenaar.com

Yesterday:

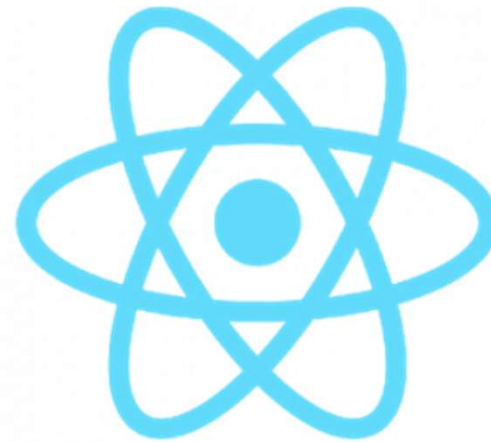
- Finish **Express** – API's and middleware
- **React in a nutshell:**
 - Concepts
 - Structure + architecture + building
 - Components (classes + functions)
 - Hooks
 - State
 - Props
 - Data

Express middleware



“Order *does* matter!”

Front-end Frameworks – the big four



React



React Basics:

- [Introduction](#) – overview of the front-end landscape
- React [tooling](#) – installation
 - Node.js / webpack & vite
 - Vite React App (`npm create vite@latest --options`)
- The [structure and architecture](#) of React apps.
 - General structure, adding components, loading components, adding functionality
- [Class based](#) vs. [Function based](#) components
- [state](#) and [props](#) in components

3 Basic principles in every React App

1. Components
2. Reactive Updates
3. Virtual DOM in memory

Important Files

- `Package.json`
- `main.jsx`
- `App.jsx`

Starting your app

`npm run dev` (or `npm start` for older, webpack applications)

does a lot of stuff for you under the covers:

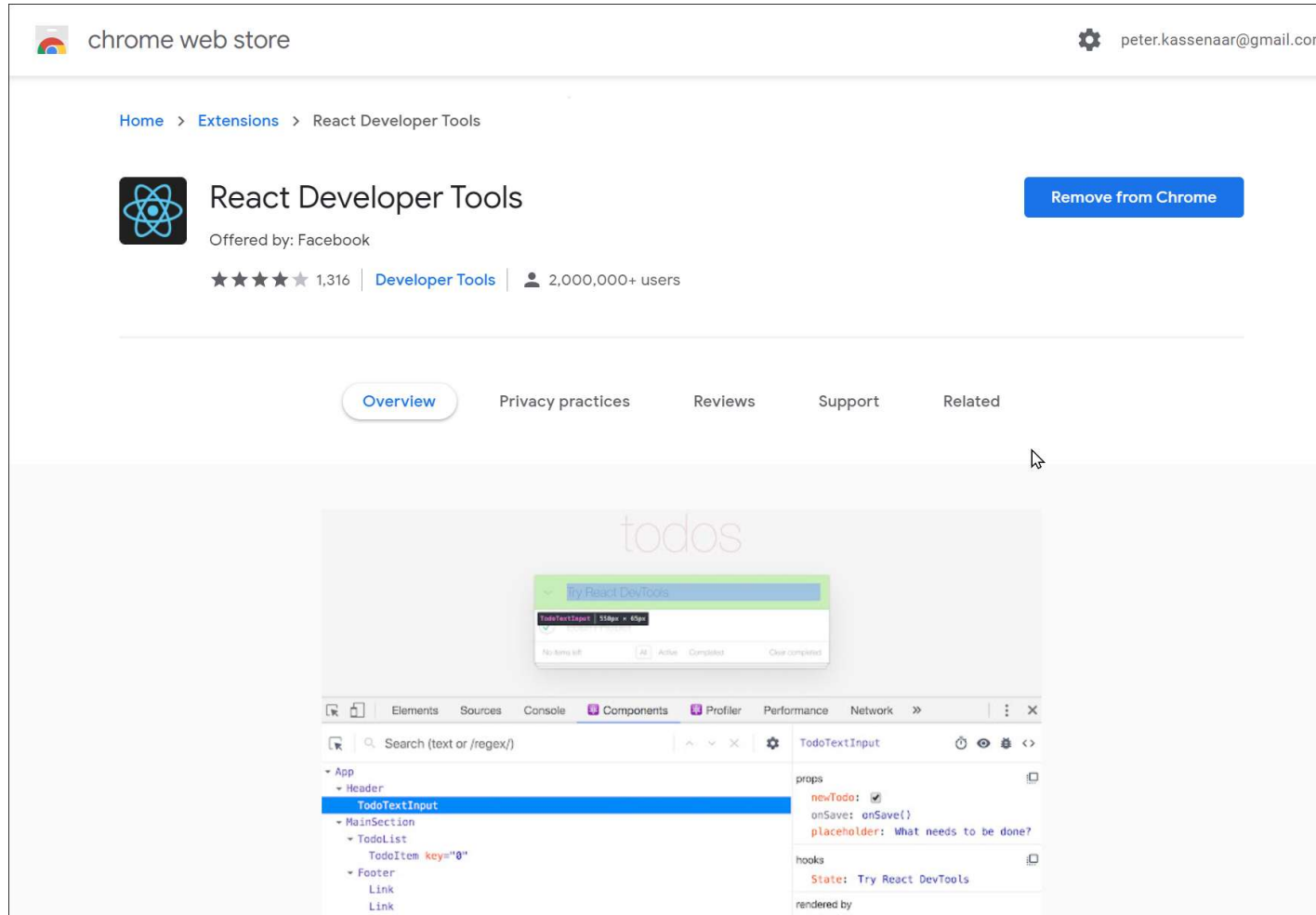
- Kicks off `Vite/Babel` for transpiling JSX
- Compile `SASS` to CSS
- Compile the `TypeScript` to JavaScript
- `Vite` or `Webpack` for bundling files
- Sets up webserver at <http://localhost:5173/>
- Sets up `Live Reload` (HMR, *Hot Module Reloading*) for better DX
- ...

Principles we covered:

- JSX (looks like HTML, but isn't)
- State
 - `const [someVar, setSomeVar]` in [functions](#)
 - `state={...}` and `setState()` in [classes](#)
- Concept: "Lifting state up"
 - Statefull- / Functional- / Smart- / [Container-components](#)
 - Stateless- / Presentational- / View- / [Dumb-components](#)
- Passing props: variables and functions
 - Inline functions to trigger state changes
`(<button onClick={() => someFunction()} ">)`

Extra – React DevTools

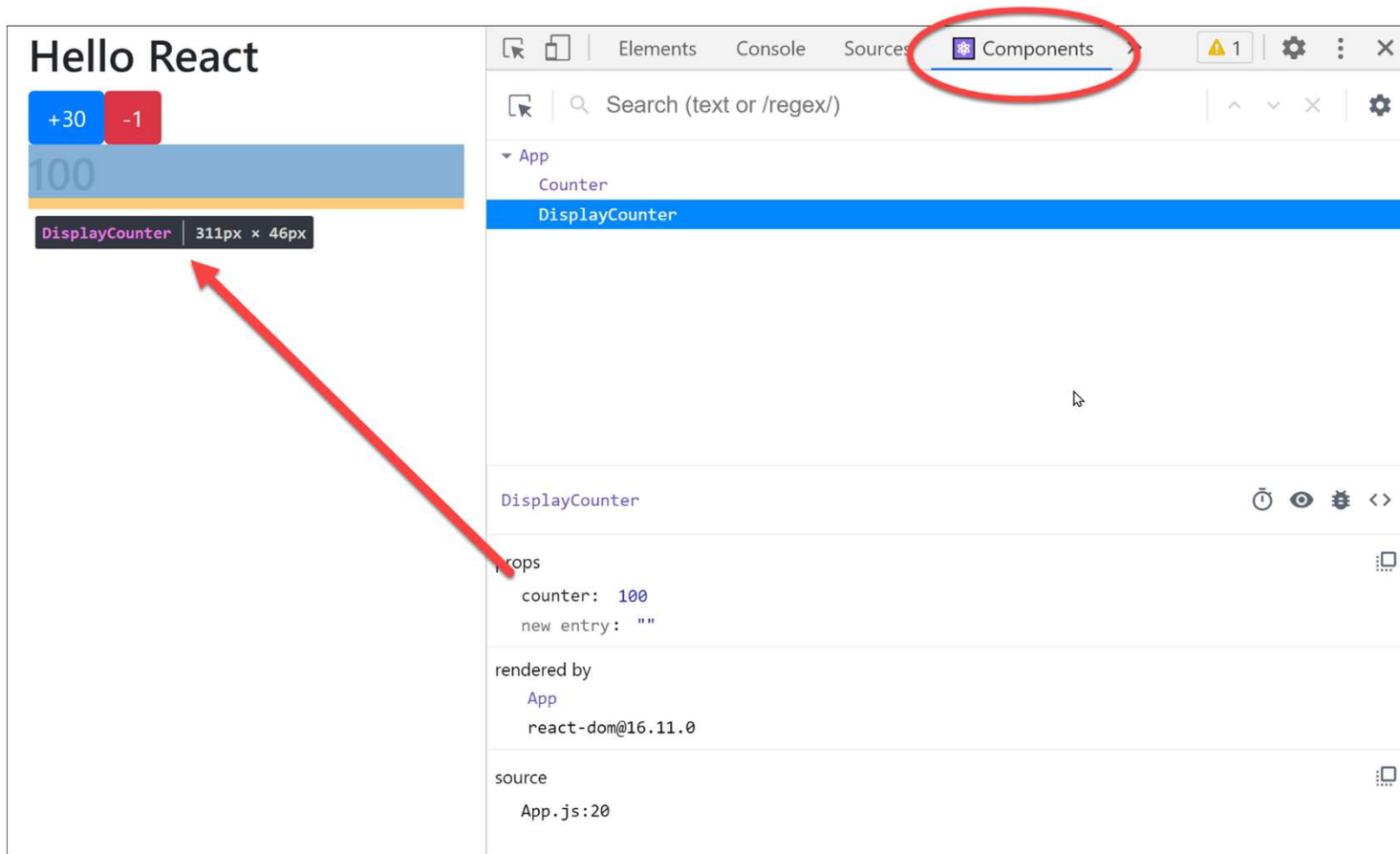
- Inspect the structure of your application in the DOM



<https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en>

Install in Chrome

- Open Chrome DevTools and choose tab **Components**



Other

Questions

Your questions

- **Q 1:** *"Multithreading, multiprocessing, and workers. When is each one useful? Is it even useful at all?"*
 - Requirement: **handle many processes in parallel** (quickly reading data from many meters, handling incoming requests from many sources at the same time), without one process having to wait for another or a long queue forming.
- **A 1:** Node.js is single-threaded but **non-blocking I/O handles concurrent requests well.**
 - The event loop is your friend for I/O-bound work.
 - You only need workers for **CPU-bound** work.

Dummy code on Workers:

- Verdict: on the meter-reading use case:
 - incoming requests are **I/O-bound**: the event loop handles thousands concurrently without workers. → No workers needed!
 - If you're **processing** the data (aggregating, transforming), offload that to a worker thread! This keeps the event loop ready for additional requests

```
// dummy code: worker_threads – CPU-heavy task off the main thread
const { Worker, isMainThread, parentPort, workerData } = require('worker_threads');

if (isMainThread) {
  const worker = new Worker(__filename, { workerData: { meterId: 42 } });
  worker.on('message', result => console.log('Result:', result));
} else {
  // transformation: run in (external) worker thread
  const result = someHeavyComputation(workerData.meterId);
  parentPort.postMessage(result);
}
```

More info: https://nodejs.org/api/worker_threads.html

AI on workers and threads:

Scenario	Tool
Many concurrent HTTP requests / DB reads	Event loop handles it — no workers needed
Heavy computation (parsing, crypto, transforms)	<code>worker_threads</code>
Fault isolation / separate memory	<code>child_process</code>
Scale across CPU cores	<code>cluster</code> module

Event Emitters

- **Q 2:** *"Event Emitters: when should we use them?"*
- **A 2: YES:**
 - Use when you have **one-to-many** notifications (example: push notification),
 - When the emitter **shouldn't know about its listeners** (example: decoupled pub/sub-system within a process).
- So, use event emitters with:
 - real-time data streams,
 - internal domain events,
 - plugin systems.
- NO: in case of simple **request/response** flows
 - just use async/await functions or Promises.

Dummy code on Event Emitters

```
// Import EventEmitter class
const {EventEmitter} = require('events');

// Define a class that extends EventEmitter and create instance
class MeterBus extends EventEmitter {}
const bus = new MeterBus();

// Multiple listeners, decoupled
// 1. Add a listener for logging
bus.on('reading', ({id, value}) => console.log(`Log: meter ${id} = ${value}`));
bus.on('reading', ({id, value}) => { // 2. Add a second listener for alerts
    if (value > 100) alertOps(id); // 3. Call alertOps if the value is above 100
});

// 4. Emit a 'reading' event with data
bus.emit('reading', {id: 'M001', value: 142});
```


Linting your Node.js code?

- **Q 3:** *LINT tools (such as ESLint/JSLint) – how relevant or useful are they in Node.js?*
- **A 3:** Short answer: **Absolutely use ESLint in Node.js.**
 - Maybe even more important than in frontend, because there's **no compiler catching your mistakes**
 - errors hit production directly (!)
- **Preferred:** add linting to CI pipeline:
 - treat errors as build failures.
 - Combine for instance with *Prettier* for formatting so ESLint stays focused on code quality.

Example .eslintrc

- Configure your eslint for example like this, for Node.js applications:

```
// .eslintrc.js – example, but sensible Node.js config
module.exports = {
  env: { node: true, es2022: true },
  extends: ['eslint:recommended'],
  rules: {
    'no-unused-vars': 'warn',
    'no-console': 'off',           // fine in Node
    'eqeqeq': 'error',
    'no-process-exit': 'warn', // optional: graceful shutdown
  }
};
```

Today:

- rendering external data – cont'd
- Rendering images
- Conditional rendering
- Forms and handling user input
 - Concept: Controlled~ vs. Uncontrolled components
- Using Http and external API's – complete Apps

Summary

- We covered:
- Node.js
 - Structure & architecture
 - Modules
 - Bundlers
 - Best Practices
- React:
 - Components
 - Hooks and (local) state
 - Communication

React Next Steps

- Building/Publishing your apps
- Styling - [React Router](#) Web/Advanced [Routing](#)
- Routing [Guards](#)
- State Management with [Context](#) and/or [Redux](#)
- [HOC](#) Components
- Writing [Custom Hooks](#)
- [Unit](#) testing
- [Error](#) handling
- ...

GitHub Repos Essential for Every React Developer

DEVPowered by Algolia

Create Post11

27 jun 2020

**Suraj Vishwakarma**
Posted on 9 sep

226 7 4 6 6

GitHub Repos Essential for Every React Developer

#react #javascript #webdev #beginners

Introduction

React is one of the most used frameworks for building frontend today. With many frameworks of React, more options are taking React to the next level. So learning React is a must when learning web development especially when you are involved in frontend development.

Last year's [StackOverflow](#) survey shows that React is the most used web framework for building frontend.



Framework	Usage Percentage
Node.js	42.65%
React	40.58%
jQuery	21.98%
Express	19.28%
Angular	17.46%
Next.js	16.67%

More from Suraj Vishwakarma

Comparing The Top React Frameworks
#webdev #javascript #beginners #react

Building an Infinite Scroll Component in React
#webdev #react #javascript #beginners

A Beginner's Guide to Starting in Technical Article Writing
#beginners #webdev #programming #writing

<https://dev.to/surajonddev/github-repos-essential-for-every-react-developer-464m>



Wrap-up

What have we learned?

Goals

1. You *will* learn about the **structure and architecture** of React Apps.
From a small `hello-world` app to the largest enterprise applications.
2. You are familiar with the **main React concepts** of the framework. You can always google the code details yourself.
3. You will have **some hands-on experience** regarding creating apps and components, API's/backends, security concepts, routing and forms.
4. You will have a **general understanding** of the way modern web apps are created using React and build tools.

Concepts – more important than code!

- Components
 - Classes, functions
- Data & State
 - `.map()`, data binding
- Props
- Forms
 - Controlled/uncontrolled components
- Http-communication

If you liked it, endorse

The screenshot shows a LinkedIn profile page. The top navigation bar includes the LinkedIn logo, a search bar, and links for Home, My Network (with a red notification badge), Jobs, Messaging, Notifications, Me, Work, and Try Premium for Free. The profile section displays the user's name, location, and education details. Below this, the 'Featured Skills & Endorsements' section is highlighted with a red circle. It lists three skills: Teaching (3 endorsements), TypeScript (3 endorsements), and Angular (2 endorsements). Each skill entry includes the number of endorsements and the names of the endorsing connections. To the right of the skills list is a button to 'Add a new skill'. Below the skills list is a link to 'See 26 more skills'. At the bottom of the profile section is the 'Recommendations' section, which includes a link to 'Ask to be recommended'. On the right side of the profile, there is a button to 'Add new profile section' and a 'Messaging' button at the bottom.

duursporten (lopen, zwemmen, fietsen, roeien, schaatsen).

Laatste jaar in combinatie met studie Bewegingswetenschappen, Rijksuniversiteit Groningen (RUG).

[See more education](#) ▾

Featured Skills & Endorsements [Add a new skill](#) ✎

Teaching · 3 Youri Pattynama and 2 connections have given endorsements for this skill

TypeScript · 3 Youri Pattynama and 2 connections have given endorsements for this skill

Angular · 2 Momen El Kamri and 1 connection have given endorsements for this skill

[See 26 more skills](#) ▾

Recommendations [Ask to be recommended](#) ✎

[Add new profile section](#)

● Messaging ✎ ⚙

<https://www.linkedin.com/in/peterkassenaar/>



Thank you